Mixed-Rank Weight Decomposition, Optimization Landscape Reshaping, and Dead Neuron Revival

Revised Notes — v2

Working Notes

March 20, 2026

Abstract

We develop the mathematical framework for understanding how decomposing a neural network weight matrix as a sum of factored terms $W = \sum_i B_i C_i$ reshapes the optimization landscape without altering the representable function class. We give the *complete* gradient analysis—including both B_i and C_i updates—showing precisely how each contributes to dead neuron revival. We carefully distinguish between SVD, sum-of-rank-1, and mixed-rank decompositions. Finally, we discuss a more general nonlinear parameterization $W = f(\theta)$ and its implications.

Contents

1 Setup and Notation **3**

2 Complete Gradient Analysis **3**
2.1 Total Change in the Effective Weight Matrix . . . 3
2.2 Deriving the Dead Neuron Weight Change Step by Step . . . 4

3 Comparing Factorizations Through the Coupling Matrix **6**
3.1 Standard Parameterization (W directly) . . . 6
3.2 SVD: $W = U\Sigma V^\top$ . . . 7
3.3 Sum of Rank-1 (Unconstrained): $W = \sum_{i=1}^{K} \mathbf{b}_i \mathbf{c}_i^\top$ . . . 7
3.4 Mixed-Rank: $W = \sum_i B_i C_i$ with Varying r_i . . . 8

4 Nonlinear Parameterization: $W = f(\theta)$ **9**
4.1 Motivation . . . 9
4.2 The Dead Neuron Condition Revisited . . . 9
4.3 Examples of Nonlinear Parameterizations . . . 9
4.3.1 Element-wise nonlinearity: $W = g(A) \cdot B + C$ . . . 9
4.3.2 Matrix exponential: $W = W_0 \exp(\Delta)$ . . . 10
4.3.3 Hyperbolic parameterization: $W = A \odot \tanh(B)$ . . . 10
4.3.4 Neural parameterization: $W = h_\psi(\mathbf{z})$ . . . 10
4.4 The General Principle . . . 11
4.5 Why Nonlinearity Could Help Beyond Bilinear . . . 11

5 Dynamic Gauge Transformations **11**

6 The Practical Formulation: $W + BC$ with Everything Trainable **12**
6.1 The Minimal Intervention 12
6.2 Relationship to LoRA and ReLoRA 13

7 Redundancy as a Feature, Not a Bug **13**
7.1 Inverting the Efficiency Paradigm 13
7.2 When Efficiency Helps and When It Hurts 14
7.3 Reinterpreting Scaling Laws 14
7.4 Connection to Continual Learning 15

8 Summary **15**

1 Setup and Notation

Consider a single feedforward layer with ReLU activation:

$$\mathbf{y} = \sigma\big(W\mathbf{x} + \mathbf{b}\big), \qquad \sigma(z) = \max(0, z), \tag{1}$$

where $W \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$.

The j-th neuron's pre-activation is $z_j = \mathbf{w}_j^\top \mathbf{x} + b_j$, where $\mathbf{w}_j^\top$ is the j-th row of W.

Definition 1 (Dead neuron)**.** *Neuron j is* dead *with respect to data distribution $\mathcal{D}$ if $z_j(\mathbf{x}) < 0$ for all $\mathbf{x} \in \operatorname{supp}(\mathcal{D})$.*

Let $\mathcal{A}$ denote the alive set and $\mathcal{D}_{\text{dead}}$ the dead set. Write the loss gradient as:

$$G \coloneqq \frac{\partial \mathcal{L}}{\partial W} \in \mathbb{R}^{m\times n}, \qquad \text{with } G_{j,\cdot} = \mathbf{0}^\top \text{ for all } j \in \mathcal{D}_{\text{dead}}. \tag{2}$$

2 Complete Gradient Analysis

We decompose:

$$W = \sum_{i=1}^{K} B_i C_i, \qquad B_i \in \mathbb{R}^{m\times r_i},\ C_i \in \mathbb{R}^{r_i\times n}. \tag{3}$$

The gradients with respect to each factor are:

$$\frac{\partial \mathcal{L}}{\partial C_i} = B_i^\top G \ \in \mathbb{R}^{r_i\times n}, \tag{4}$$

$$\frac{\partial \mathcal{L}}{\partial B_i} = G\, C_i^\top \ \in \mathbb{R}^{m\times r_i}. \tag{5}$$

After one gradient step with learning rate η:

$$C_i' = C_i - \eta\, B_i^\top G, \tag{6}$$

$$B_i' = B_i - \eta\, G\, C_i^\top. \tag{7}$$

2.1 Total Change in the Effective Weight Matrix

The new effective weight is:

$$W' = \sum_{i=1}^{K} B_i' C_i' = \sum_{i=1}^{K} \big(B_i - \eta\, G C_i^\top\big)\big(C_i - \eta\, B_i^\top G\big). \tag{8}$$

Expanding and collecting by order in η:

$$W' = \underbrace{\sum_i B_i C_i}_{=W} - \eta \underbrace{\sum_i \Big[B_i\big(B_i^\top G\big) + \big(G C_i^\top\big) C_i\Big]}_{\text{first-order: } \Delta W^{(1)}} + \eta^2 \underbrace{\sum_i \big(G C_i^\top\big)\big(B_i^\top G\big)}_{\text{second-order: } \Delta W^{(2)}}. \tag{9}$$

Therefore the total change in the effective weight is:

$$\boxed{\Delta W = -\eta \sum_{i=1}^{K} \Big[B_i B_i^\top G + G C_i^\top C_i\Big] + \eta^2 \sum_{i=1}^{K} G C_i^\top B_i^\top G.} \tag{10}$$

2.2 Deriving the Dead Neuron Weight Change Step by Step

We now extract the row of ΔW corresponding to dead neuron $j \in \mathcal{D}_{\text{dead}}$, showing every intermediate step.

Step 1: Extract row j. Left-multiply ΔW from (10) by $\mathbf{e}_j^\top$ (the j-th standard basis row vector):

$$\Delta \mathbf{w}_j^\top \;=\; \mathbf{e}_j^\top \Delta W \;=\; -\eta \sum_{i=1}^{K} \Big[\underbrace{\mathbf{e}_j^\top B_i B_i^\top G}_{\text{Term 1 (“}C\text{-channel”)}} \;+\; \underbrace{\mathbf{e}_j^\top G\, C_i^\top C_i}_{\text{Term 2 (“}B\text{-channel”)}} \Big] \;+\; O(\eta^2). \tag{11}$$

We handle each term separately.

Step 2: Evaluate Term 2 (B-channel). The factor $\mathbf{e}_j^\top G$ extracts row j of G:

$$\mathbf{e}_j^\top G \;=\; G_{j,\cdot} \;\in \mathbb{R}^{1\times n}. \tag{12}$$

For dead neuron j, we have $G_{j,\cdot} = \mathbf{0}^\top$ by definition. Therefore:

$$\mathbf{e}_j^\top G\, C_i^\top C_i \;=\; \underbrace{G_{j,\cdot}}_{=\mathbf{0}^\top} C_i^\top C_i \;=\; \mathbf{0}^\top \qquad \text{for all } i. \tag{13}$$

The B-channel is dead for dead neurons

$$\boxed{\text{Term 2:} \quad \sum_{i=1}^{K} \mathbf{e}_j^\top G\, C_i^\top C_i = \mathbf{0}^\top \quad \text{for } j \in \mathcal{D}_{\text{dead}}.} \tag{14}$$

Why: Updating B_i requires the gradient $GC_i^\top$ (Eq. 5). Row j of this matrix is $G_{j,\cdot}C_i^\top$, which involves the dead neuron's own gradient row. Since that row is zero, the B_i update cannot change dead neuron j's weights.

Step 3: Evaluate Term 1 (C-channel). We expand $B_i^\top G$ by inserting a resolution of the identity $\sum_{k=1}^{m} \mathbf{e}_k \mathbf{e}_k^\top = I$:

$$B_i^\top G \;=\; B_i^\top \Big(\sum_{k=1}^{m} \mathbf{e}_k\, \mathbf{e}_k^\top \Big) G \;=\; \sum_{k=1}^{m} \big(B_i^\top \mathbf{e}_k\big)\big(\mathbf{e}_k^\top G\big) \;=\; \sum_{k=1}^{m} \big(B_i^\top \mathbf{e}_k\big)\, G_{k,\cdot}. \tag{15}$$

Since $G_{k,\cdot} = \mathbf{0}^\top$ for $k \in \mathcal{D}_{\text{dead}}$, only alive neurons contribute:

$$B_i^\top G \;=\; \sum_{k\in\mathcal{A}} \big(B_i^\top \mathbf{e}_k\big)\, G_{k,\cdot}. \tag{16}$$

Now left-multiply by $\mathbf{e}_j^\top B_i$:

$$\mathbf{e}_j^\top B_i B_i^\top G \;=\; \big(\mathbf{e}_j^\top B_i\big) \sum_{k\in\mathcal{A}} \big(B_i^\top \mathbf{e}_k\big)\, G_{k,\cdot} \;=\; \sum_{k\in\mathcal{A}} \underbrace{\big(\mathbf{e}_j^\top B_i\big)\big(\mathbf{e}_k^\top B_i\big)^\top}_{\text{scalar: see Step 4}} G_{k,\cdot}. \tag{17}$$

Step 4: Identify the coupling coefficient. The quantity $\mathbf{e}_j^\top B_i \in \mathbb{R}^{1\times r_i}$ is the j-th row of B_i. Similarly $\mathbf{e}_k^\top B_i$ is the k-th row. Their product is:

$$\big(\mathbf{e}_j^\top B_i\big)\big(\mathbf{e}_k^\top B_i\big)^\top \;=\; \sum_{\ell=1}^{r_i} [B_i]_{j\ell}\,[B_i]_{k\ell} \;=\; \big[B_i B_i^\top\big]_{jk}. \tag{18}$$

This is a *scalar* — the (j,k) entry of the PSD matrix $B_i B_i^\top$.

Step 5: Sum over all terms i. Substituting Step 4 into (17) and summing over i:

$$\sum_{i=1}^{K} \mathbf{e}_j^\top B_i B_i^\top G \;=\; \sum_{k\in\mathcal{A}} \underbrace{\left(\sum_{i=1}^{K} \big[B_i B_i^\top\big]_{jk}\right)}_{\coloneqq\ \kappa_{jk}} G_{k,\cdot}. \tag{19}$$

Step 6: Assemble the final result. Combining Steps 2 and 5 in Eq. (11):

$$\Delta \mathbf{w}_j^\top = -\eta\left[\sum_{k\in\mathcal{A}} \kappa_{jk}\, G_{k,\cdot} \;+\; \mathbf{0}^\top\right] + O(\eta^2). \tag{20}$$

Step 7: Verify the second-order term also vanishes. From (10), the second-order term for row j is:

$$\big[\Delta W^{(2)}\big]_{j,\cdot} \;=\; \eta^2 \sum_i \mathbf{e}_j^\top \big(G C_i^\top\big)\big(B_i^\top G\big) \;=\; \eta^2 \sum_i \underbrace{\big(G_{j,\cdot}\, C_i^\top\big)}_{=\mathbf{0}^\top\ (\text{dead } j)} B_i^\top G \;=\; \mathbf{0}^\top. \tag{21}$$

The second-order term vanishes for the same reason as Term 2: it requires $G_{j,\cdot} \neq \mathbf{0}$.

Complete Revival Formula (all orders)

For dead neuron $j \in \mathcal{D}_{\text{dead}}$, the total weight change through order η^2 is:

$$\boxed{\Delta \mathbf{w}_j^\top \;=\; -\eta \sum_{k \in \mathcal{A}} \kappa_{jk}\, G_{k,\cdot},} \tag{22}$$

where the **coupling matrix** is the positive semi-definite matrix:

$$\kappa_{jk} \;=\; \left[\sum_{i=1}^{K} B_i B_i^\top\right]_{jk} \;=\; \sum_{i=1}^{K} \sum_{\ell=1}^{r_i} [B_i]_{j\ell}\, [B_i]_{k\ell}. \tag{23}$$

Interpretation: The weight change of dead neuron j is a weighted sum of the alive neurons' gradients, with weights given by κ_{jk}. Revival occurs if and only if this weighted sum is nonzero, i.e., if $\sum_{k \in \mathcal{A}} \kappa_{jk}\, G_{k,\cdot} \neq \mathbf{0}^\top$.

Where each piece comes from:

- $G_{k,\cdot}$ — gradient signal from alive neuron k (the "information source").
- $[B_i]_{k\ell}$ — how strongly alive neuron k projects into factor i (the "transmitter").
- $[B_i]_{j\ell}$ — how strongly dead neuron j projects into factor i (the "receiver").
- The product $[B_i]_{j\ell}[B_i]_{k\ell}$ — the coupling strength through channel ℓ of factor i.
- Summing over ℓ and i aggregates all coupling channels.

Remark 1. *Define $P := \sum_i B_i B_i^\top \in \mathbb{R}^{m \times m}$. This PSD matrix determines the entire coupling structure. The decomposition affects dead neurons* only *through P. Different decompositions of the same W can yield different P, and this freedom is what enables revival.*

3 Comparing Factorizations Through the Coupling Matrix

All factorizations of the form (3) produce revival governed by the same coupling matrix $P = \sum_i B_i B_i^\top$. The factorizations differ in *what P looks like* and *how much freedom you have in shaping it.*

3.1 Standard Parameterization (W directly)

No factorization. Formally $K = 1$, $B_1 = I$, $C_1 = W$:

$$P = I \cdot I^\top = I. \tag{24}$$

The coupling matrix is the identity: $\kappa_{jk} = \delta_{jk}$. Dead neuron j couples only to itself. Since its own gradient is zero, **no revival occurs**.

Actually, this requires care. With $B_1 = I$ (fixed, not learned) and $C_1 = W$ (learned), the C-update gives $\Delta W = -\eta\, I \cdot I^\top G = -\eta G$, which is just standard gradient descent. The diagonal coupling means no cross-neuron information flow.

3.2 SVD: $W = U\Sigma V^\top$

This is a **product** factorization, not a sum. We can rewrite it as a sum of rank-1 terms:

$$W = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top, \tag{25}$$

which fits our framework with $B_i = \sigma_i \mathbf{u}_i \in \mathbb{R}^{m\times 1}$ and $C_i = \mathbf{v}_i^\top \in \mathbb{R}^{1\times n}$.

The coupling matrix is:

$$P_{\text{SVD}} = \sum_{i=1}^{r} \sigma_i^2 \, \mathbf{u}_i \mathbf{u}_i^\top = U\Sigma^2 U^\top. \tag{26}$$

The coupling between dead neuron j and alive neuron k:

$$\kappa_{jk}^{\text{SVD}} = \sum_{i=1}^{r} \sigma_i^2 \, [\mathbf{u}_i]_j \, [\mathbf{u}_i]_k. \tag{27}$$

The problem. Since U is orthogonal, $UU^\top = I$, so *if* we keep all singular values equal ($\sigma_i = \sigma$ for all i), then $P = \sigma^2 I$, recovering diagonal coupling. In practice, the σ_i are unequal, so $P_{\text{SVD}} \neq \alpha I$. But the orthogonality of U constrains the coupling:

$$\sum_{k=1}^{m} \kappa_{jk}^{\text{SVD}} = \sum_i \sigma_i^2 [\mathbf{u}_i]_j \underbrace{\sum_k [\mathbf{u}_i]_k}. \tag{28}$$

The columns of U have unit norm and are orthogonal, so the coupling is **spectrally constrained**. Neurons aligned with the top singular vectors get strong coupling; neurons in the null space get none. There is **no freedom to choose** P—it is fully determined by the current W.

3.3 Sum of Rank-1 (Unconstrained): $W = \sum_{i=1}^{K} \mathbf{b}_i \mathbf{c}_i^\top$

Here $B_i = \mathbf{b}_i \in \mathbb{R}^{m\times 1}$, $C_i = \mathbf{c}_i^\top \in \mathbb{R}^{1\times n}$, with $K \geq r = \text{rank}(W)$.

The coupling matrix is:

$$P_{\text{rank1}} = \sum_{i=1}^{K} \mathbf{b}_i \mathbf{b}_i^\top. \tag{29}$$

Key difference from SVD. The vectors $\mathbf{b}_i$ are **not required to be orthogonal**. This means:

(i) We can have $K > r$, i.e., *redundant* terms. Redundancy gives freedom to shape P.

(ii) The vectors $\mathbf{b}_i$ can be dense, giving dense off-diagonal entries in P.

(iii) Multiple $\mathbf{b}_i$ can point in similar directions, *amplifying* coupling in those directions.

Example 1. *Suppose $m = 3$, neuron 3 is dead, neurons 1 and 2 are alive.*

- ***SVD:*** *If $\mathbf{u}_1 = (1,0,0)^\top$, $\mathbf{u}_2 = (0,1,0)^\top$, then $\kappa_{31} = \kappa_{32} = 0$. Neuron 3 gets no coupling. We cannot change this without changing W.*

- ***Sum of rank-1:*** *We can choose* $\mathbf{b}_1 = (1,0,1)^\top$, $\mathbf{b}_2 = (0,1,1)^\top$ *and solve for* $\mathbf{c}_1, \mathbf{c}_2$ *to match* W. *Now* $\kappa_{31} = 1$, $\kappa_{32} = 1$*: neuron 3 is coupled to both alive neurons.* ***Same*** W***, different*** P***.***

The Freedom in Sum-of-Rank-1

For a given W, the SVD gives a *unique* coupling matrix $P = U\Sigma^2 U^\top$. The sum-of-rank-1 decomposition allows *any* $P \succeq 0$ that is consistent with $W = \sum_i \mathbf{b}_i \mathbf{c}_i^\top$. This freedom is the gauge freedom, and it is what enables us to choose coupling structures favorable for revival.

3.4 Mixed-Rank: $W = \sum_i B_i C_i$ with Varying r_i

The mixed-rank case adds another dimension of control. With terms of different ranks r_i:

$$P_{\text{mixed}} = \sum_{i=1}^{K} B_i B_i^\top. \tag{30}$$

Each term $B_i B_i^\top$ is a rank-r_i PSD matrix. The mixed-rank structure allows:

Low-rank terms (r_i small): $B_i \in \mathbb{R}^{m \times r_i}$ with $r_i \ll m$. The matrix $B_i B_i^\top$ has rank r_i, so its action is concentrated on a r_i-dimensional subspace. But its *entries can be dense*: all m neurons project into this low-dimensional space, creating strong inter-neuron coupling.

High-rank terms (r_i large): $B_i B_i^\top$ has rank up to $r_i \sim m$. It can approximate a diagonal or near-diagonal matrix, providing neuron-specific capacity without strong coupling.

The mixture.

$$P_{\text{mixed}} = \underbrace{\sum_{i \in \text{low}} B_i B_i^\top}_{\text{dense, low-rank: coupling}} + \underbrace{\sum_{i \in \text{high}} B_i B_i^\top}_{\text{near-diagonal, high-rank: capacity}}. \tag{31}$$

This is not achievable with a pure sum-of-rank-1 decomposition unless K is very large (many rank-1 terms to approximate both coupling and capacity roles), nor with SVD (no freedom in P).

Factorization	**Coupling matrix P**	**Freedom in P**	**Revival control**
Standard (W)	I	None	None
SVD	$U\Sigma^2 U^\top$	None (fixed by W)	None
Sum of rank-1	$\sum_i \mathbf{b}_i \mathbf{b}_i^\top$	High	Strong, uniform
Mixed-rank	$\sum_i B_i B_i^\top$	Highest	Strong, tunable

4 Nonlinear Parameterization: $W = f(\theta)$

4.1 Motivation

The linear decomposition $W = \sum_i B_i C_i$ is a *linear* function of the parameters $\theta = \{B_i, C_i\}$ (bilinear, to be precise: linear in each B_i given C_i and vice versa). But nothing requires the parameterization map to be linear.

Definition 2 (General reparameterization)**.** *Let $\theta \in \Theta$ be a parameter vector and $f : \Theta \to \mathbb{R}^{m\times n}$ a differentiable map. We define:*

$$W = f(\theta), \qquad \widetilde{\mathcal{L}}(\theta) = \mathcal{L}\big(f(\theta)\big). \tag{32}$$

The gradient in θ-space is:

$$\nabla_\theta \widetilde{\mathcal{L}} = \big[Df(\theta)\big]^\top \operatorname{vec}(G), \tag{33}$$

where $Df(\theta) \in \mathbb{R}^{mn\times \dim(\Theta)}$ is the Jacobian. The effective update in W-space is:

$$\Delta W = -\eta\, Df(\theta)\big[Df(\theta)\big]^\top \operatorname{vec}(G) + O(\eta^2). \tag{34}$$

The matrix $\mathbf{M}(\theta) \coloneqq Df(\theta)[Df(\theta)]^\top \in \mathbb{R}^{mn\times mn}$ acts as a **preconditioning matrix** (equivalently, the pullback metric).

4.2 The Dead Neuron Condition Revisited

For dead neuron j, the rows of G indexed by j are zero, i.e., $\operatorname{vec}(G)$ has zeros in positions $\{(j-1)n+1, \ldots, jn\}$. Whether ΔW has nonzero entries at the corresponding positions depends on whether $\mathbf{M}(\theta)$ mixes those positions with the nonzero positions.

Proposition 1 (General revival condition)**.** *Dead neuron j receives a nonzero update if and only if the preconditioner $\mathbf{M}(\theta)$ has nonzero entries coupling the "alive" positions to the "dead" positions in $\operatorname{vec}(W)$. Formally, let $\mathcal{I}_j = \{(j-1)n+1, \ldots, jn\}$ and $\mathcal{I}_\mathcal{A} = \bigcup_{k\in\mathcal{A}} \mathcal{I}_k$. Revival occurs if:*

$$\mathbf{M}(\theta)[\mathcal{I}_j, \mathcal{I}_\mathcal{A}] \neq \mathbf{0}. \tag{35}$$

For the linear case $f(\theta) = \sum_i B_i C_i$, this reduces to the coupling matrix $P = \sum_i B_i B_i^\top$ from Section 3. For a general nonlinear f, the preconditioner can have much richer structure.

4.3 Examples of Nonlinear Parameterizations

4.3.1 Element-wise nonlinearity: $W = g(A) \cdot B + C$

Let g be an element-wise nonlinearity (e.g., tanh, sigmoid). Then:

$$\frac{\partial W_{jk}}{\partial A_{jk}} = g'(A_{jk}) \cdot B_{jk}, \qquad \frac{\partial W_{jk}}{\partial B_{jk}} = g(A_{jk}), \qquad \frac{\partial W_{jk}}{\partial C_{jk}} = 1. \tag{36}$$

The Jacobian mixes entries in a pattern determined by $g'(A)$ and B. Even if $G_{j,\cdot} = \mathbf{0}$, the nonlinearity in g means that the *curvature* of f can create coupling at second order that is absent in the linear case.

4.3.2 Matrix exponential: $W = W_0 \exp(\Delta)$

Define $f(\Delta) = W_0 \exp(\Delta)$ where Δ is a learnable perturbation. The matrix exponential couples all entries:

$$\frac{\partial}{\partial \Delta} W_0 e^{\Delta} = W_0 \int_0^1 e^{s\Delta} \, (\cdot) \, e^{(1-s)\Delta} \, ds, \tag{37}$$

where $(\cdot)$ denotes the slot for the perturbation direction. This integral mixes all rows and columns of Δ, providing dense coupling even for infinitesimal updates. Dead rows of G would be spread across all entries of Δ through the exponential's Taylor series.

4.3.3 Hyperbolic parameterization: $W = A \odot \tanh(B)$

The Hadamard product with a saturating nonlinearity creates position-dependent scaling:

$$\frac{\partial \mathcal{L}}{\partial B_{jk}} = G_{jk} \cdot A_{jk} \cdot \big(1 - \tanh^2(B_{jk})\big). \tag{38}$$

Here, B_{jk} near saturation ($|\tanh(B_{jk})| \approx 1$) has vanishing gradient regardless of G_{jk}, introducing a *new* kind of absorbing state. This illustrates that nonlinear parameterizations can create pathologies as well as cure them.

4.3.4 Neural parameterization: $W = h_\psi(\mathbf{z})$

Use a small neural network h_ψ that takes a fixed input $\mathbf{z}$ (e.g., positional encoding of the layer index) and outputs the weight matrix. This is a form of **hypernetwork**. The coupling structure is determined by the hypernetwork's architecture:

$$\frac{\partial \mathcal{L}}{\partial \psi} = \left(\frac{\partial \, \mathrm{vec}(W)}{\partial \psi} \right)^{\top} \mathrm{vec}(G). \tag{39}$$

If h_ψ has shared hidden layers, then *all* entries of W are coupled through ψ, providing maximal mixing. Dead neurons' weights change whenever any alive neuron's gradient is nonzero.

4.4 The General Principle

Design Principle for Reparameterization

Given a parameterization $W = f(\theta)$:

(i) Compute the preconditioner $\mathbf{M}(\theta) = Df(\theta)[Df(\theta)]^\top$.

(ii) Check that $\mathbf{M}$ has nonzero off-diagonal blocks coupling dead-neuron rows to alive-neuron rows.

(iii) Ensure that $\mathbf{M}$ does not introduce *new* absorbing states (e.g., through saturation).

The "ideal" nonlinear parameterization would have:

- Dense coupling (no neuron is isolated) — for revival.
- Tunable coupling strength — so coupling doesn't destabilize training.
- No new absorbing states — the cure should not introduce new diseases.
- Efficient computation — the overhead of f and Df should be manageable.

4.5 Why Nonlinearity Could Help Beyond Bilinear

In the bilinear case ($W = \sum_i B_i C_i$), we showed that all revival comes through the C-channel at first order; the B-channel contributes nothing to dead neurons. This is because the B-gradient involves $GC_i^\top$, and the dead rows of G kill the contribution.

A nonlinear f can break this symmetry. If f mixes entries of θ nonlinearly, then the Jacobian Df can couple dead-row positions to alive-row positions even through the "B-like" parameters.

Example 2. *Consider $W = A + \phi(B)^\top \phi(B)$ where $\phi : \mathbb{R}^{r\times m} \to \mathbb{R}^{r\times m}$ is a nonlinear map applied row-wise to B. The gradient with respect to B:*

$$\frac{\partial \mathcal{L}}{\partial B} = 2\, D\phi(B)^\top \big[\phi(B)\, G^\top + \phi(B)\, G\big]. \tag{40}$$

The term $\phi(B)G^\top$ involves all rows of $\phi(B)$—not just the rows indexed by alive neurons—because $\phi(B)$ lives in a different space ($\mathbb{R}^{r\times m}$) and the matrix multiplication mixes all rows of G. This means dead-neuron positions receive gradients through B that they would not receive in the bilinear case.

5 Dynamic Gauge Transformations

At any training step t:

1. Collapse: $W^{(t)} = \sum_i B_i^{(t)} C_i^{(t)}$ (or $= f(\theta^{(t)})$).
2. Re-decompose: find new $\widetilde{\theta}$ such that $f(\widetilde{\theta}) = W^{(t)}$.
3. Continue training from $\widetilde{\theta}$.

What is preserved: $f(\widetilde{\theta}) = W^{(t)}$, so the function is identical.

What changes: The preconditioner $\mathbf{M}(\widetilde{\theta}) \neq \mathbf{M}(\theta^{(t)})$, hence:

- Different gradient directions in W-space.
- Different coupling patterns between alive and dead neurons.
- Absorbing states at $\theta^{(t)}$ are generically non-absorbing at $\widetilde{\theta}$.

For nonlinear parameterizations, finding $\widetilde{\theta}$ such that $f(\widetilde{\theta}) = W^{(t)}$ may require solving an inverse problem, which could be non-trivial. For the bilinear case, re-decomposition is straightforward (e.g., random B_i, solve C_i by least squares).

6 The Practical Formulation: $W + BC$ with Everything Trainable

6.1 The Minimal Intervention

The simplest realization of the coupling mechanism requires no restructuring of W. Keep the standard weight matrix and add a small low-rank correction:

$$W_{\text{eff}} = W + BC, \qquad B \in \mathbb{R}^{m\times r},\ C \in \mathbb{R}^{r\times n},\ r \ll \min(m,n), \tag{41}$$

where W, B, and C are **all trainable**. Initialize $C = 0$ (or B random, $C = 0$) so that $W_{\text{eff}} = W$ at the start—no change to the function.

Gradients.

$$\frac{\partial \mathcal{L}}{\partial W} = G, \qquad \frac{\partial \mathcal{L}}{\partial B} = GC^\top, \qquad \frac{\partial \mathcal{L}}{\partial C} = B^\top G. \tag{42}$$

Dead neuron j's effective weight change. Following the analysis of Section 2:

$$\Delta[\mathbf{w}_{\text{eff}}]_j^\top = \underbrace{-\eta\, G_{j,\cdot}}_{=\mathbf{0}^\top \text{ (from } W)} + \underbrace{\mathbf{0}^\top}_{B\text{-channel dead}} - \eta \sum_{k\in\mathcal{A}} [BB^\top]_{jk}\, G_{k,\cdot}. \tag{43}$$

The first two terms vanish. The third is the coupling channel from the C-update, identical to our general formula with coupling matrix $P = I + BB^\top$.

Memory cost.

$$\text{Parameters:} \quad mn + r(m+n). \tag{44}$$

For $m = n = 4096$ and $r = 10$: 16.8M + 81.9K, an overhead of $< 0.5\%$. Even with Adam ($3\times$ for optimizer states), the coupling channel costs almost nothing.

Periodic reset. At any step: merge $W \leftarrow W + BC$, reset $C \leftarrow 0$, optionally re-randomize B. The function is unchanged, the coupling directions are refreshed, and no optimizer surgery on W's states is needed.

6.2 Relationship to LoRA and ReLoRA

The structure $W_{\text{eff}} = W + BC$ is identical to LoRA in form but fundamentally different in intent and execution.

LoRA freezes W and trains only B, C. The purpose is **parameter efficiency**: reduce the number of trainable parameters for fine-tuning. Since W receives no gradient updates, the W-channel is entirely shut off. LoRA has no mechanism for dead neuron revival—the frozen W keeps dead neurons permanently dead.

ReLoRA extends LoRA to pre-training by periodically merging BC into W and resetting. Critically, W is **still frozen between merges**—only B and C are trainable during each phase. The motivation is **rank accumulation**: successive low-rank updates sum to a high-rank total update. ReLoRA has no theory of gradient coupling or dead neuron revival.

Our proposal keeps W **trainable at all times** alongside B and C. The low-rank term is not carrying the update—W does the normal learning via standard gradient descent. The BC term exists purely as a **coupling channel** that breaks absorbing states. The motivation is not efficiency but **optimization geometry**.

	LoRA	**ReLoRA**	**Ours**
W trainable?	No	No (between merges)	**Yes (always)**
BC role	Carries the update	Carries the update	**Coupling channel**
Periodic merge?	No	Yes + optimizer reset	Yes (simpler)
Motivation	Parameter efficiency	Rank accumulation	**Break absorbing states**
Dead neuron theory?	None	None	**Yes** ($P = I + BB^\top$)

Empirical prediction from the theory. ReLoRA has been shown to *fail* for small language models (11M–66M parameters), where it underperforms full-rank training and amplifies rank deficiencies. This is precisely what our framework predicts: freezing W removes the direct gradient pathway, which disproportionately hurts small models where every neuron's contribution matters. Our approach keeps that pathway open and adds coupling on top.

7 Redundancy as a Feature, Not a Bug

7.1 Inverting the Efficiency Paradigm

The entire LoRA/ReLoRA/GaLore literature asks one question: *how do we reduce parameters while preserving performance?* The implicit assumption is that fewer parameters is better, and the goal is to find the minimum parameterization that still works.

Our framework inverts this:

The Core Insight

We are not parameter-efficient. We are deliberately **parameter-inefficient** through redundancy. The redundancy does not increase the function class—it makes the *same* function class **easier to optimize**.

The extra parameters in BC purchase something that cannot be obtained by any amount of training on W alone: **access to regions of function space that gradient descent on W cannot reach** due to absorbing states.

This is distinct from the well-known benefit of overparameterization in the function class (e.g., wider networks are easier to train). That kind of overparameterization changes *what* the network can represent. Our proposal is overparameterization of the *parameterization map*, not the function class: same representable functions, more paths to reach them.

7.2 When Efficiency Helps and When It Hurts

The relationship between model size and the need for redundancy is:

	Large models	**Small models**
Dead neuron proportion	Low	High
Effective capacity / nominal capacity	≈ 1	$\ll 1$
Optimization landscape	Well-behaved	Pathological
Benefit of parameter efficiency	High (save memory)	Low (nothing to save)
Benefit of parameter redundancy	Low (already healthy)	**High (breaks pathology)**

The implication:

> *Large models can afford to be parameter-efficient because their optimization landscape is already well-behaved. Small models need to be parameter-inefficient because their optimization landscape is pathological.*

7.3 Reinterpreting Scaling Laws

Standard scaling laws model performance as a function of architectural capacity:

$$\text{Performance} = f(N), \tag{45}$$

where N is the number of parameters. But if small models are optimization-limited rather than capacity-limited, the actual relationship is:

$$\text{Performance} = f(N) \cdot g(N), \tag{46}$$

where $g(N) \leq 1$ is an **optimization efficiency factor** that degrades for small N (due to dead neurons, rank collapse, softmax bottleneck).

The scaling laws *as measured* conflate f and g. They attribute to insufficient capacity what is actually insufficient optimization. Evidence for this interpretation:

(i) Distillation consistently improves small model performance beyond what training from scratch achieves at the same model size—the soft targets smooth the optimization landscape, effectively increasing $g(N)$ without changing $f(N)$.

(ii) Small models exhibit saturation (loss *increases* in late training)—this is optimization pathology (g decreasing), not capacity limitation.

(iii) The Phi model family achieves anomalously high performance at small scale through data quality interventions—better data increases $g(N)$ by providing more informative gradients.

Our framework provides a structural intervention: the redundant parameterization pushes $g(N) \to 1$ directly, without requiring a teacher model (distillation) or specially curated data (Phi).

Testable Prediction

A small model trained with the $W + BC$ parameterization (all trainable, periodic resets, $r \ll m$) should:

(i) Avoid the saturation phenomenon observed in standard small model training.

(ii) Achieve loss levels closer to what scaling laws predict for a *larger* model.

(iii) Approach distillation-level performance without requiring a teacher model.

(iv) Show fewer dead neurons and higher effective rank throughout training.

The improvement should be greatest for the smallest models on the longest training runs, where cumulative optimization pathology is most severe.

7.4 Connection to Continual Learning

The same principle applies across tasks. In continual learning, the network's effective capacity degrades over the task sequence as neurons die and representations degenerate. Existing methods fight this with defensive strategies (regularization, replay, frozen subnetworks).

The redundant parameterization provides an alternative: the coupling channels enable **automatic capacity recovery** between tasks. Dead neurons from task A are revived by gradients from task B. No explicit capacity management is needed—the optimization dynamics handle it.

At task boundaries, the periodic reset ($W \leftarrow W + BC$, $C \leftarrow 0$, re-randomize B) refreshes the coupling directions without altering the learned function. This is a natural intervention point that aligns with the task structure.

8 Summary

1. The complete gradient analysis shows that dead neuron revival comes **entirely through the C-channel** at first order: the B-update contributes nothing to dead neurons in the bilinear case. The coupling is governed by $P = \sum_i B_i B_i^\top$.

2. **SVD** gives no freedom in P (it is fixed by W). **Sum-of-rank-1** gives freedom through non-orthogonal $\mathbf{b}_i$. **Mixed-rank** gives the most freedom, allowing separate tuning of coupling (low-rank terms) and capacity (high-rank terms).

3. **Nonlinear parameterizations** $W = f(\theta)$ generalize the framework and can provide coupling through *both* factor-update channels, not just the C-channel. The preconditioner $\mathbf{M}(\theta) = Df\, Df^\top$ is the unified object governing revival across all parameterizations.

4. The choice of parameterization is a **gauge choice**: it changes the optimization trajectory without changing the function class. Dynamic re-parameterization during training exploits this gauge freedom to escape absorbing states.

5. The most practical realization is $W_{\text{eff}} = W + BC$ with all parameters trainable and periodic resets. This has $< 0.5\%$ memory overhead, requires no architectural changes, and is compatible with existing training infrastructure. Unlike LoRA and ReLoRA, the low-rank term serves as a **coupling channel**, not an update carrier.

6. **Redundancy is the mechanism, not a cost.** The extra parameters do not expand the function class—they make the same class easier to optimize by breaking absorbing states. This inverts the prevailing efficiency paradigm: small models need *more* parameterization (of the right kind), not less.

7. Current scaling laws conflate **capacity limitations** with **optimization limitations**. The redundant parameterization targets the optimization factor directly, predicting that small models can exceed their apparent scaling-law ceiling without architectural changes, distillation, or curated data.